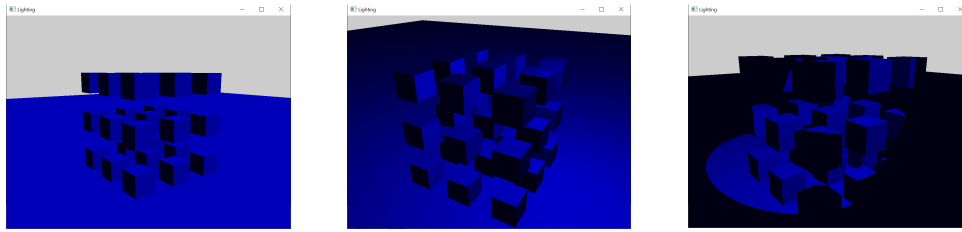


COMP3011 Computer Graphics



Spring 2026

Lab 7

Learning Outcomes

After Lab 7 you are expected to be able to

1. Specify vertex normals correctly.
2. Set up VAOs and shader variables required for lighting calculations.
3. Recognise that fragment positions in the fragment shader are in the viewport space, therefore an additional variable is required to store the fragment position in world space for lighting calculations.
4. Implement ambient, diffuse and specular and combine them in an implementation of the phong lighting model.
5. Use the phong lighting model to implement three well-known light types, directional, positional, and spot light.
6. Define and use a function in the fragment shader.
7. Recognise the effect of attenuation on per fragment lighting.
8. Demonstrate the ability to program per fragment lighting calculations.
9. Use some useful GLSL functions including `normalize()`, `max()`, `pow()`, `radians()`, `cos()`, `reflect()`.
10. Write a fragment shader for per fragment lighting.

Lighting

In the COMP3011 directory create a new project called

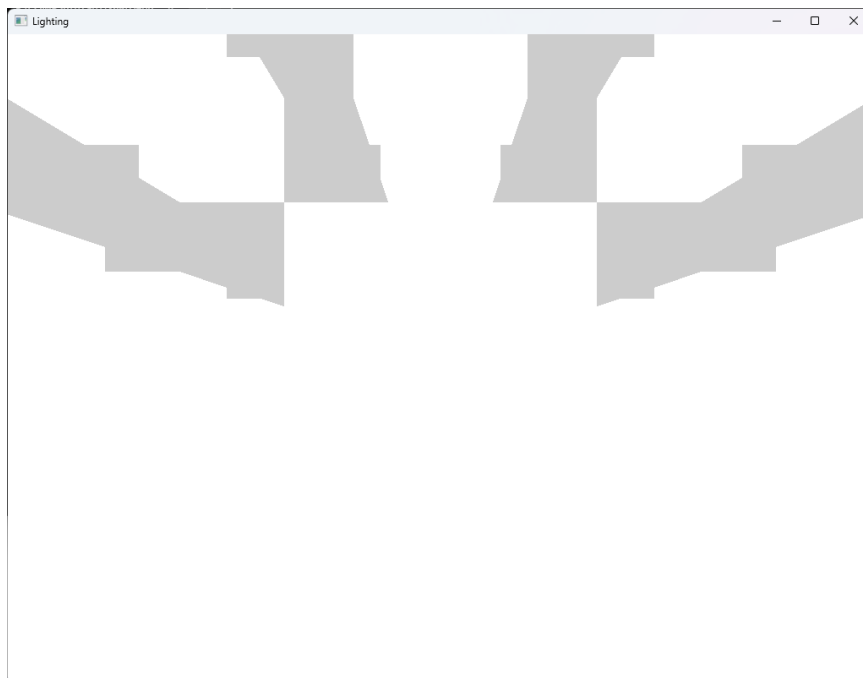
Lab7

Open Lab7.cpp on Moodle and copy the code to your new project.

Specify the location of the include and lib directories, add the glfw3.lib input dependency, and add gl3w.c to the source files.

Download phong.vert and phong.frag from Moodle, copy them to your project directory, and add them to the Source Files in the Solution Explorer.

If you run the program, then the following window should appear



Add your implementation of the `processKeyboard()` function from lab6a, i.e. the Model-Viewer camera, to your code. If you haven't implemented that yet then you'll need to do Lab6 before you do this lab.

Run the program and move the camera to a location like this



You will see that there is a 3x3x3 array of cubes and a floor. There is no light and all of the faces of each cube are white.

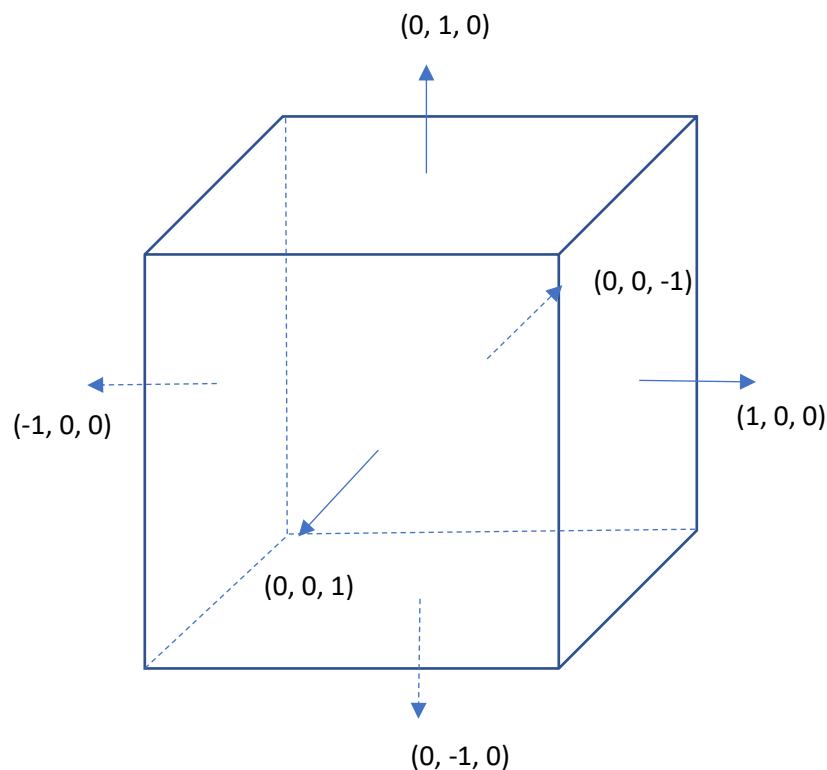
In this lab we will add lights to this scene and test several lighting implementations.

Directional light

Each vertex, of each triangle, on each face, needs to have a normal vertex attribute specified in the vertex array.

```
float vertices[] =  
{  
    //back face  
    //pos          //col          //normal  
    -0.5f, -0.5f, -0.5f, 1.f, 1.0f, 1.0f,  
    0.5f, -0.5f, -0.5f, 1.f, 1.0f, 1.0f,  
    0.5f, 0.5f, -0.5f, 1.f, 1.0f, 1.0f,  
    0.5f, 0.5f, -0.5f, 1.f, 1.0f, 1.0f,  
    -0.5f, 0.5f, -0.5f, 1.f, 1.0f, 1.0f,  
    -0.5f, -0.5f, -0.5f, 1.f, 1.0f, 1.0f,  
  
    //front face  
    -0.5f, -0.5f, 0.5f, 1.0f, 1.0f, 1.0f,  
    0.5f, -0.5f, 0.5f, 1.0f, 1.0f, 1.0f,  
    0.5f, 0.5f, 0.5f, 1.0f, 1.0f, 1.0f,  
    0.5f, 0.5f, 0.5f, 1.0f, 1.0f, 1.0f,  
    -0.5f, 0.5f, 0.5f, 1.0f, 1.0f, 1.0f,  
    -0.5f, -0.5f, 0.5f, 1.0f, 1.0f, 1.0f,  
}
```

Each of the faces need to have a normal vector as follows, and each vertex on that face has the same normal vector.



Write the code in the vertex array in Lab7.cpp for each vertex normal.

Next setup the VAO so that the vertex shader interprets the vertex attributes properly; 3 floats for position, 3 floats for colour, and 3 floats for normal, out of a total of 9 floats.

Create a variable to store the light direction vector

```
15
16 //light direction variable here
17 glm::vec3 lightDirection = glm::vec3(0.1f, -.81f, -.61f);
18
19 SCamera Camera;
```

In the render loop just before the code for creating the view matrix, copy the following vectors to uniforms in the shaders: light direction, light colour, camera position. Note that these uniforms haven't been declared in the shaders yet, we will do that in the next section.

```
while (!glfwWindowShouldClose(window))
{
    static const GLfloat bgd[] = { .8f, .8f, .8f, 1.f };
    glClearBufferfv(GL_COLOR, 0, bgd);
    glClear(GL_DEPTH_BUFFER_BIT);
    glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);

    glUseProgram(program);

    glUniform3f(glGetUniformLocation(program, "lightDirection"), lightDirection.x, lightDirection.y, lightDirection.z);
    glUniform3f(glGetUniformLocation(program, "lightColour"), 0.f, 0.f, 1.f);
    glUniform3f(glGetUniformLocation(program, "camPos"), Camera.Position.x, Camera.Position.y, Camera.Position.z);

    glm::mat4 view = glm::mat4(1.f);
```

In the vertex shader, add a new input vertex attribute for the normal and a new output variable for the normal. After the fragment shader has been called, in the rendering pipeline, the fragment position is in viewport space. We need to know the fragment position in world space in the fragment shader to calculate the illumination, so add an output variable for the fragment position in world space. See below.

In the vertex shader `main()` copy the vertex normal to the output variable with the additional transformations `transpose()` and `inverse()` shown below. The additional transformations ensure that the normal remains perpendicular to the surface. There is an appendix accompanying the lecture on lighting in case you are interested in how these transformations are derived. Also, transform the vertex position to world space using the model matrix.

```
#version 450 core
```

```
layout(location = 0) in vec4 vPos;  
layout(location = 1) in vec3 vCol;  
layout(location = 2) in vec3 vNor;
```

```
uniform mat4 model;  
uniform mat4 view;  
uniform mat4 projection;
```

```
out vec3 col;  
out vec3 nor;  
out vec3 FragPosWorldSpace;
```

```
void main()  
{  
    gl_Position = projection * view * model * vPos;  
    col = vCol;  
    nor = mat3(transpose(inverse(model))) * vNor;  
    FragPosWorldSpace = vec3(model * vPos);  
}
```

In the fragment shader create 2 new input variables to match the vertex shader output variables for the normal and the fragment position in world space. Add three vec3 uniforms,

1. the light colour,
2. the light direction,
3. the camera position

so that the variable names match the code you added to the render loop earlier.

```
#version 450 core

layout (location = 0) out vec4 fColour;

in vec3 col;
in vec3 nor;
in vec3 FragPosWorldSpace;

uniform vec3 lightDirection;
uniform vec3 lightColour;
uniform vec3 camPos;

void main()
{
    fColour = vec4(col, 1.f);
}
```

In the fragment shader `main()`, implement the **direction light** as it is defined on page 5 in chapter 13 and the pseudocode below.

```

ambient = k

Nnor = Normalise surface normal input
Nto_light = Negate and Normalise light direction uniform
Perform dot product Nnor with Nto_light, and ensure the result is
positive
diffuse = result of dot product

Nfrom_light = Normalise light direction uniform
NrefLight = Reflect Nfrom_light on Nnor
camDirection = camera position uniform - fragment position in world
space input variable
NcamDirection = Normalise camDirection
Perform dot product of NcamDirection with rLight, ensure result is
positive, raise to the power of shininess
specular = result of dot product

phong = ambient + diffuse + specular

```

Note that the phong, ambient, diffuse and specular terms are all floating point variables.

When implementing ambient use 0.1 as the constant.

When implementing diffuse remember that the light direction is pointing from the light, so it is negated so that it is pointing towards the light.

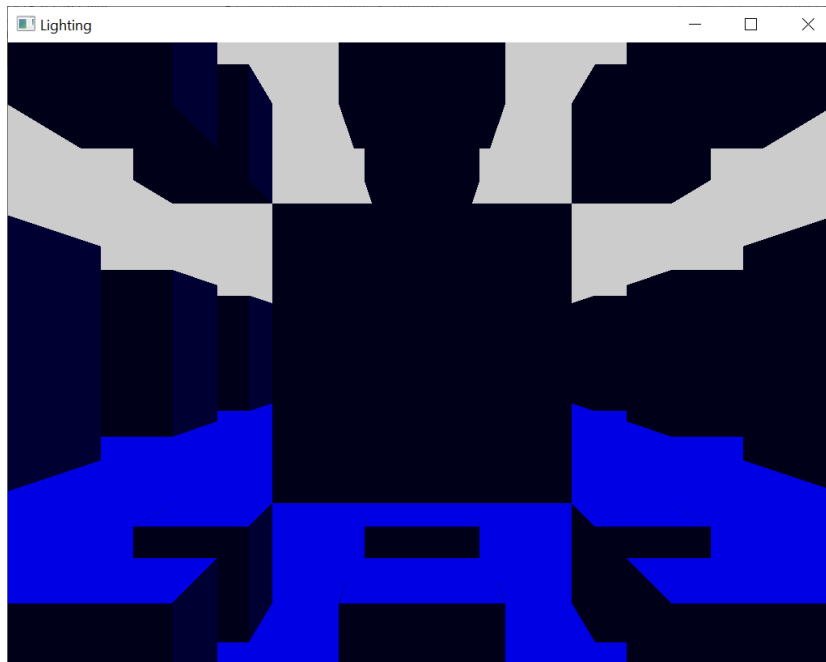
When implementing specular use 128 for the shininess value.

Useful GLSL functions are, to check a number is non-negative use `max()`, to normalise use `normalize()`, to reflect use the glsl function `reflect()`, to raise a number to a power use the glsl function `pow()`.

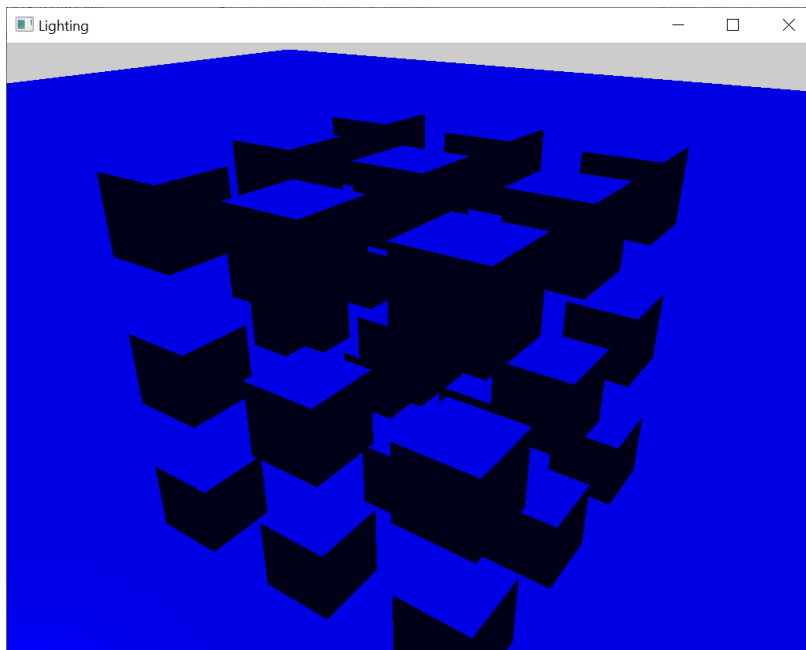
At the bottom of `main()` in the fragment shader, use the formula for calculating the colour of a fragment from the lecture notes.

```
fColour = vec4(phong * col * lightColour, 1.f);
```

If you run your program you should see this



Move the camera to a better position and you should see something like this



You can see that the top surfaces of all of the geometry are lit by a blue light.

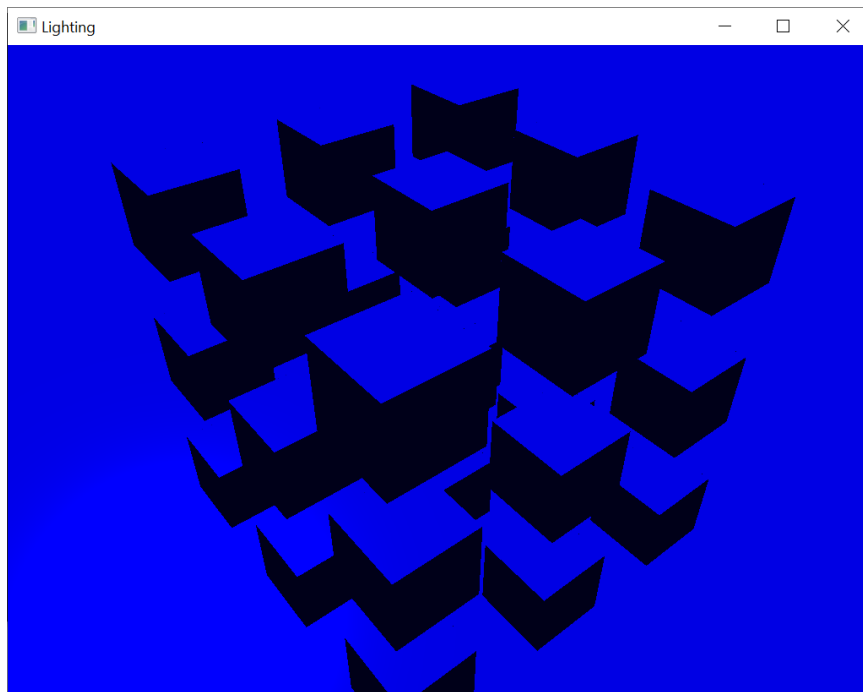
Write some code in `processKeyboard()` to set the light direction to point in the same direction the camera is currently pointing when you press the space key.

```
void processKeyboard(GLFWwindow* window)
{
    if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
        glfwSetWindowShouldClose(window, true);

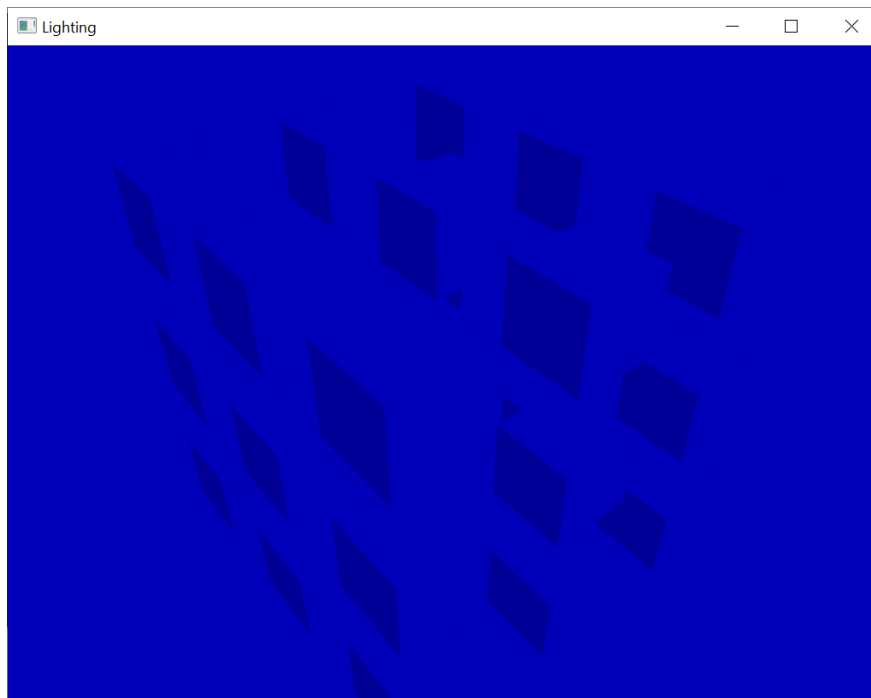
    if (glfwGetKey(window, GLFW_KEY_SPACE) == GLFW_PRESS)
    {
        light_dir = Camera.Front;
    }

    float x_off = 0.f;
    float y_off = 0.f;
    bool cam_changed = false;
    if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS)
    {
```

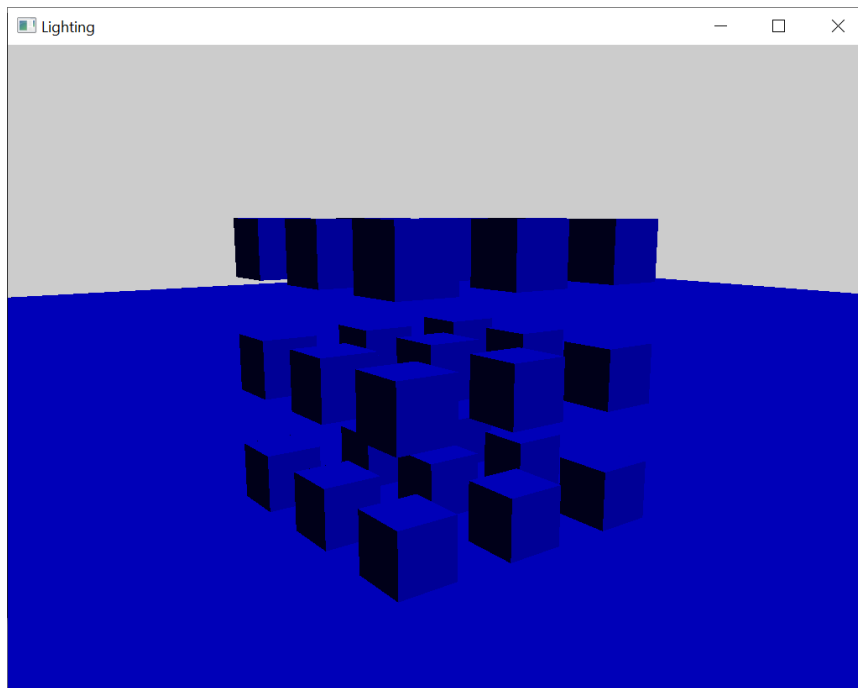
Run the program and move the camera up and back to see this



Press the space bar to set the directional light to point in the direction of the camera,



then move the camera to a different position to see the effect of the directional light, like this



Notice how all the fragments facing the same direction are illuminated exactly the same regardless of their position in the scene, because a directional light does not have a position.

Congratulations! You have now implemented a direction light, including some quite sophisticated lighting code in the fragment shader and have demonstrated the ability to program per fragment lighting.

Demo – show the program to a member of the teaching team.

Positional Light

Declare a function in the fragment shader, directly above `main()`.

```
float CalculateDirectionalIllumination()
```

Move the code for the directional light calculations from `main()` to the new function, and return the phong illumination value, then use it in the `main()` function.

```
void main()
{
    float phong = CalculateDirectionalIllumination();

    fColour = vec4(phong * col * lightColour, 1.f);
}
```

Run your program to make sure the directional lighting calculations still work.

Now we are going to implement a positional light.

Start by adding a new function to the fragment shader just above `main()`

```
float CalculatePositionalIllumination()
```

Copy and paste the code from `CalculateDirectionalIllumination()` to `CalculatePositionalIllumination()`.

Change the code in `CalculatePositionalIllumination()` so that it is an implementation of a **Positional Light** according to chapter 13 on page 7 and the pseudocode below

Ambient is the same as Directional Light

Diffuse is the same as Directional Light except

$N_{ToLight} = \text{normalize}(\text{lightPos} - \text{FragPosWorldSpace})$

Specular is the same as Directional Light except

$N_{FromLight} = -N_{ToLight}$

calculate attenuation

$\text{phong} = (\text{ambient} + \text{diffuse} + \text{specular}) * \text{attenuation}$

When calculating diffuse we are going to use a new uniform called `lightPos`, which we will declare soon.

When calculating attenuation, use the glsl function `length()` to calculate the distance between the light and the fragment, and set constant attenuation to 1.0, linear attenuation to 0.05, and quadratic attenuation to 0.002.

Ensure you use the attenuation value when calculating phong.

Add a new uniform for light position to the fragment shader

```
7 uniform vec3 lightColour;  
8 uniform vec3 lightDirection;  
9 uniform vec3 lightPos;  
10 uniform vec3 camPos;  
11
```

Declare a variable in Lab7.cpp for storing the light position

```
16 //light direction variable here  
17 glm::vec3 lightDirection = glm::vec3(0.1f, -.81f, -.61f);  
18 glm::vec3 lightPos = glm::vec3(2.f, 6.f, 7.f);  
19  
--
```

Copy the light position to the shader in the render loop

```

glUseProgram(program);

glUniform3f(glGetUniformLocation(program, "lightDirection"), lightDirection.x, lightDirection.y, lightDirection.z);
glUniform3f(glGetUniformLocation(program, "lightColour"), 0.f, 0.f, 1.f);
glUniform3f(glGetUniformLocation(program, "camPos"), Camera.Position.x, Camera.Position.y, Camera.Position.z);
glUniform3f(glGetUniformLocation(program, "lightPos"), lightPos.x, lightPos.y, lightPos.z);

```

In `processKeyboard()` set the light position to the camera position when the space bar is pressed

```

void processKeyboard(GLFWwindow* window)
{
    if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
        glfwSetWindowShouldClose(window, true);

    if (glfwGetKey(window, GLFW_KEY_SPACE) == GLFW_PRESS)
    {
        light_dir = Camera.Front;
        light_pos = Camera.Position;
    }

    float x_off = 0.f;
    float y_off = 0.f;
    bool cam_changed = false;
    if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS)

```

Call the appropriate function in your fragment shader to calculate the positional light illumination

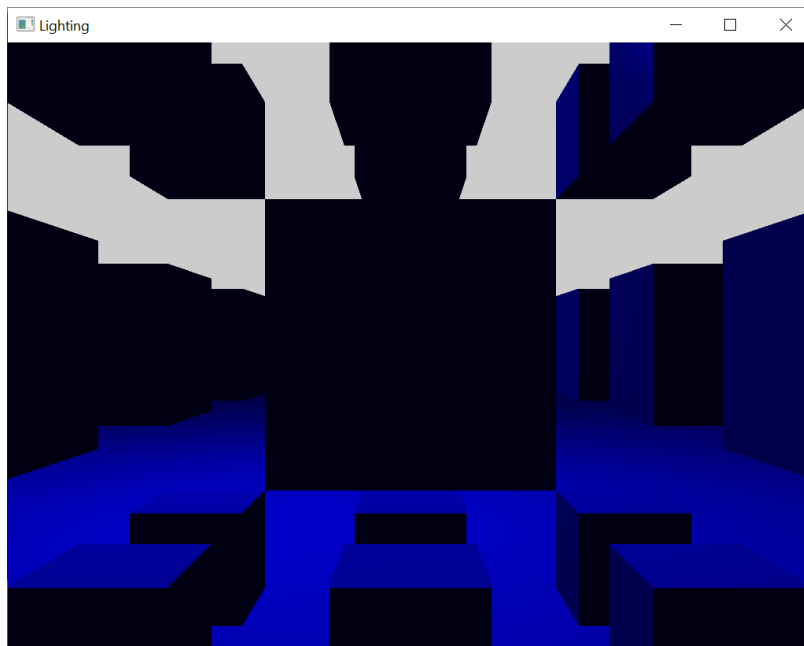
```

void main()
{
    float phong = CalculatePositionalIllumination();

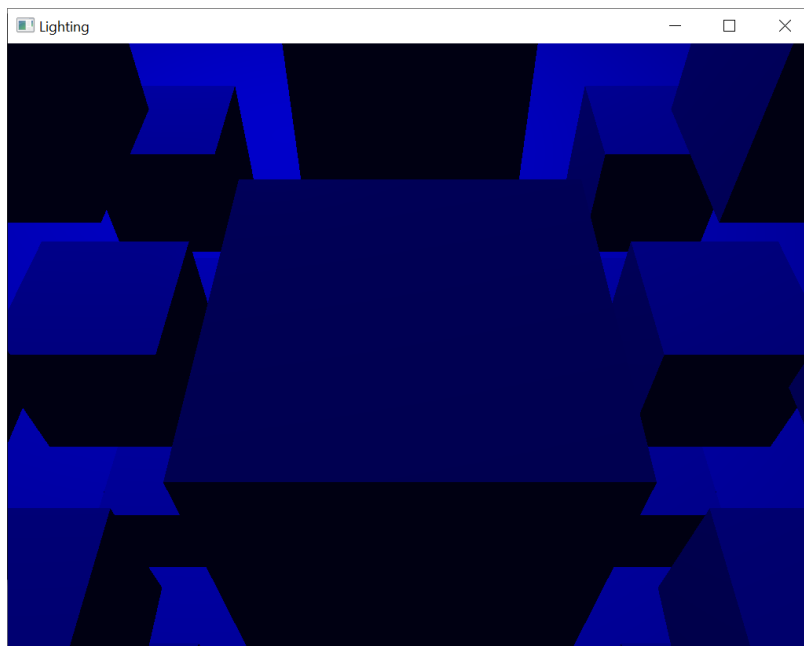
    fColour = vec4(phong * col * lightColour, 1.f);
}

```

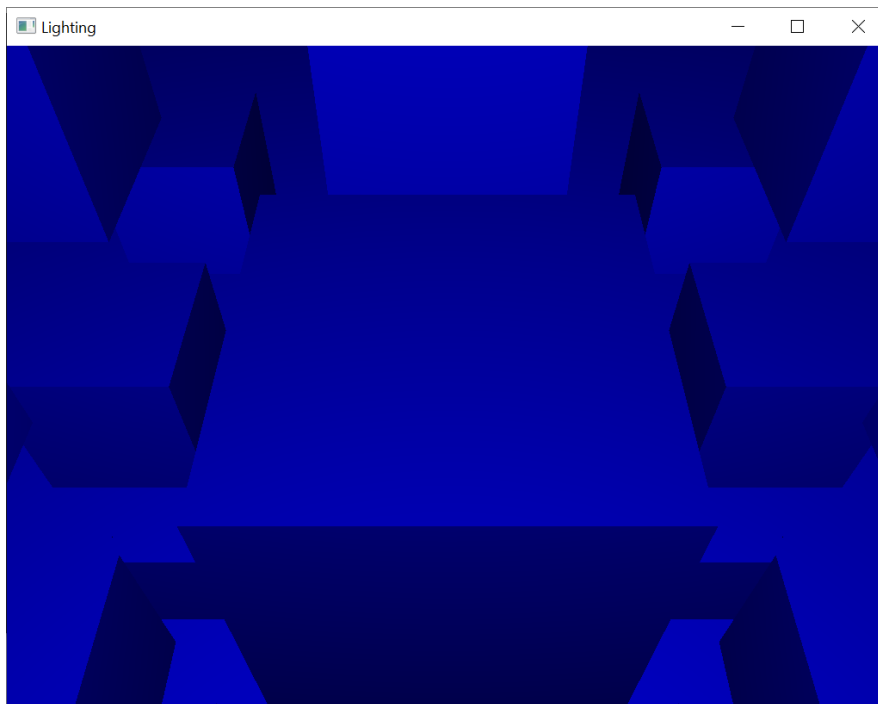
Run the program and you should see this



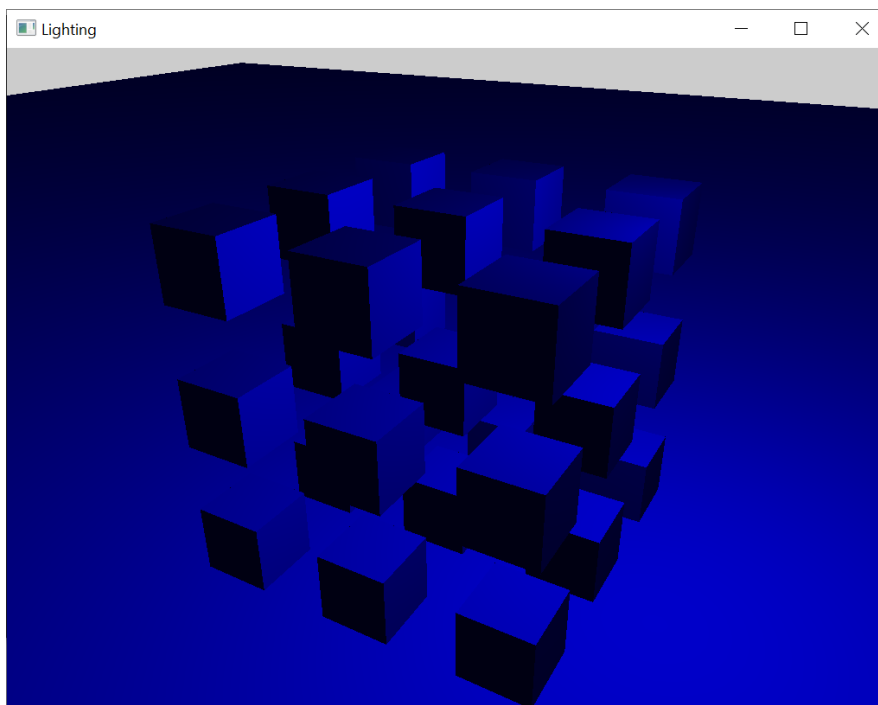
Move the camera to above one of the top row cubes



Press space bar to reposition the light.



Move the camera to a different position to see the effect of the positional light



Notice how the top face of the cube closest to where the light is positioned, where your camera was positioned when you pressed the space bar, is illuminated most brightly compared to the same faces on the other cubes. This is due to the attenuation.

Congratulations! You have now implemented a positional light.

Demo – show the program to a member of the teaching team.

Spot Light

Now we are going to implement a spot light.

Start by adding a new function to the fragment shader just above `main()`

```
float CalculateSpotIllumination()
```

Copy and paste the code from `CalculatePositionalIllumination()` to `CalculateSpotIllumination()`.

Implement the **Spot Light** code in `CalculateSpotIllumination()` according to chapter 13 on page 9 and the pseudocode below

```
Ambient is the same as Positional Light

Diffuse is the same as Positional Light

Specular is the same as Positional Light

Attenuation is the same as Positional Light

phi = convert cut_off angle to radians and calculate cosine
NSpotDir = normalize lightDirection uniform
theta = dot product of NFromLight and NSpotDir

if(theta > phi)
{
    phong = (ambient + diffuse + specular) * attenuation
}
else
{
    phong = ambient * attenuation
}
```

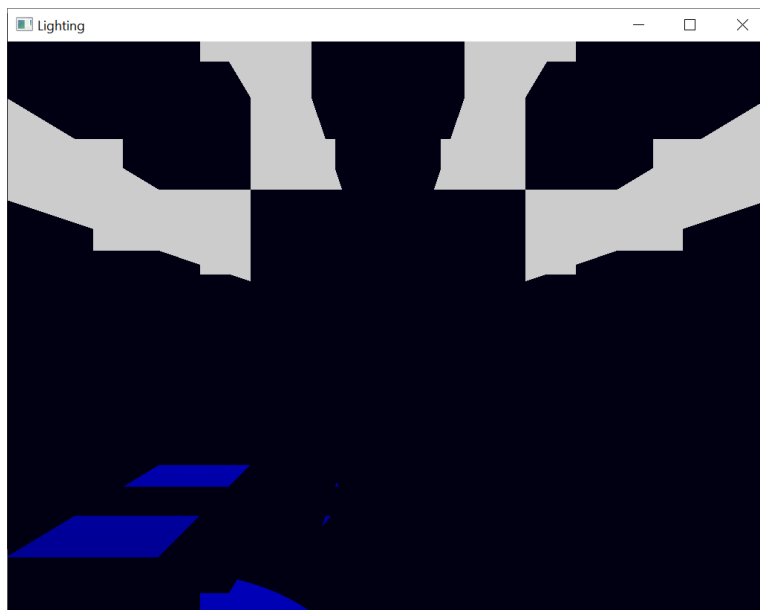
When you calculate ϕ use the glsl functions `cos()` and `radians()` and a cut off angle of 15.0.

Call the new function in your fragment shader to calculate the spot light illumination

```
void main()
{
    float phong = CalculateSpotIllumination();

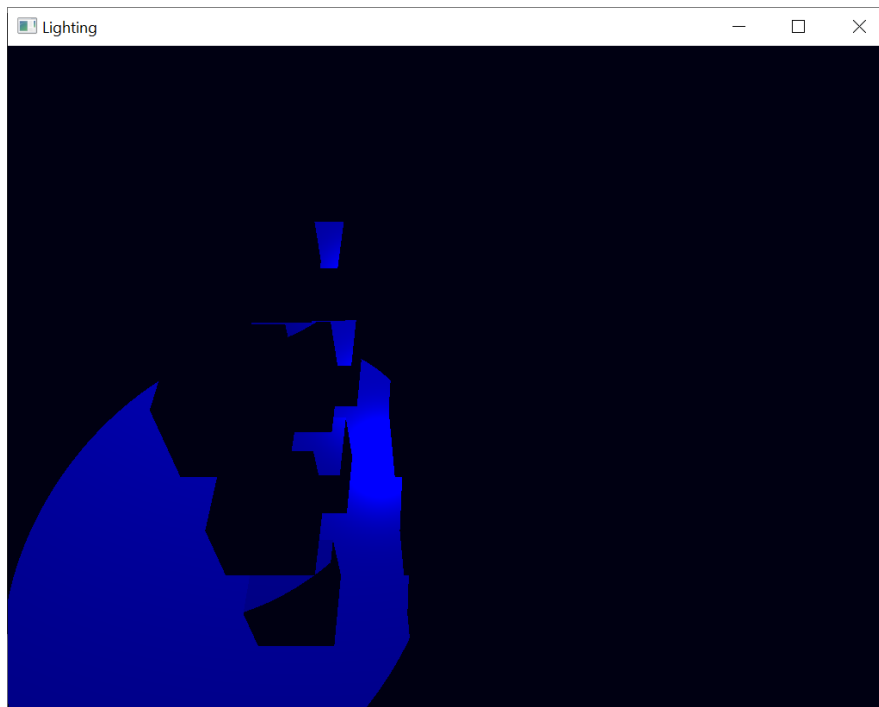
    fColour = vec4(phong * col * lightColour, 1.f);
}
```

Run the program and you should see this

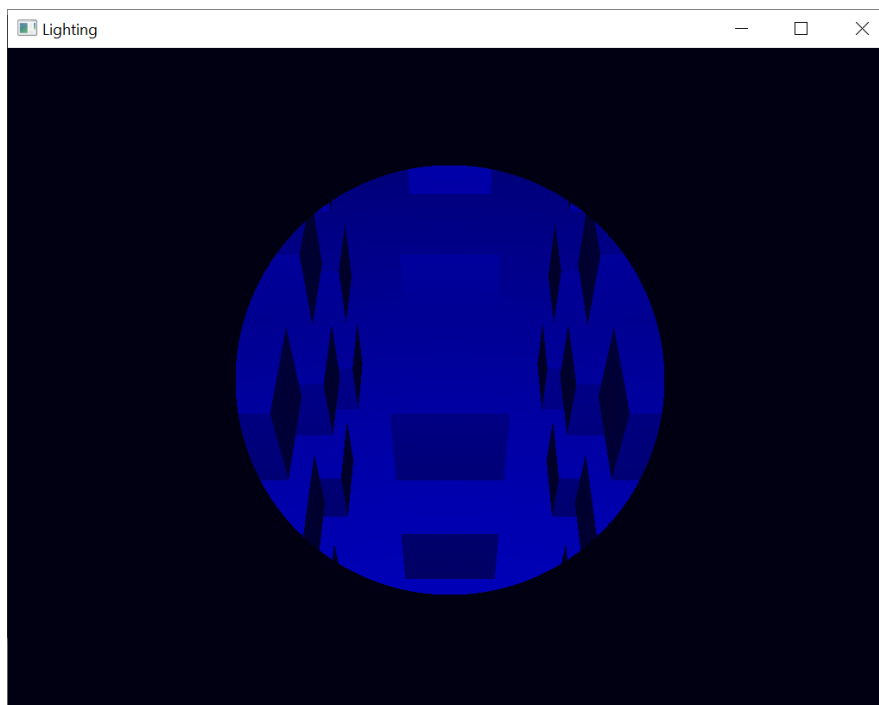


You can just make out the spot light illuminating some cubes on the left.

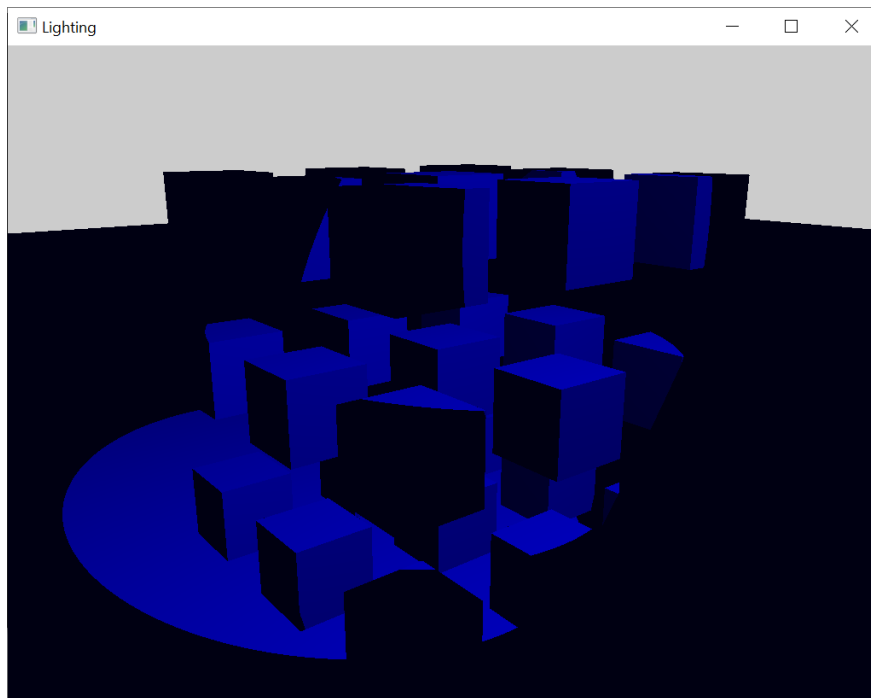
Move the camera up and backwards to a position like this



Press space bar to position and direct the spot light so it matches the camera position and direction.



Then move the camera to a new position to see the spot light effect.



Congratulations! You have now implemented a spot light.

Demo – show the program to a member of the teaching team.